Mixed effects models

Emma James

Introduction

In this practical, we will have a go at using a mixed effects model to analyse data from a factorial experiment.

Mixed effects models first became popular in psycholinguistics, where researchers want to generalise their findings beyond their specific sample of people *and* the specific words that they selected to include in their experiment. Nowadays, many areas of psychology use mixed effects models to analyse their data.

We will use a psycholinguistic dataset today, which I (Emma) collected together with Professors Lisa Henderson and Gareth Gaskell. We were interested in how children's memory for new words changes over sleep. We taught children the names of unusual animals and plants, and tested their memory after 12 hours using a picture naming task. In one condition, this 12 hours was over the day and so participants spent the time awake. In the other condition, this 12 hours spanned the night and so participants spent the time asleep. We will use a subset of the data for demonstration purposes today, but you can read more about the study here if you are interested.

Learning objectives

As a reminder, this week's learning objectives are to:

1. Describe some limitations of ANOVA and regression
2. Explain the benefits of modelling variation within and between groups
3. Define and distinguish between fixed and random effects
4. Interpret the key components and output of a basic mixed effects model
5. Evaluate the benefits and challenges associated with mixed effects modelling

---

Setting up

Create a script

Start with the usual set-up steps for the practical sessions:

1. Open your ARM_Practicals.Rproj file in RStudio
2. Create a new script and save it in your scripts/ folder as **ARM3_mixed-effects.R**
3. Download the data for the Week 3 practical from the VLE, and save it in your data/ folder.

> Tip
>
> If you don't have this folder on your current machine, repeat the set-up steps outlined in the Set-up steps from Week 1.

Load packages

We will need the following packages for today's session, so add these lines of code at the top of your script. Run them so that they are ready to use. If you don't have the packages already installed on your device, you will need to install them first using `install.packages()` as we have in previous weeks.

```
library(tidyverse)  # for data wrangling
library(lme4)       # for mixed effects models
library(lmerTest)   # for p-values
```

R defaults to scientific notation for very small numbers, which can be hard to interpret if you're not used to them. You can add this line of code to switch it off:

```
options(scipen = 999)
```

Load trial-level data

Now read in the data from your data/ folder.

```
sleep_dat <- read.csv("./data/ARM3_data_sleep.csv")
```

The dataset consists of the following variables:

- **ppt_id:** a unique identifier for each participant
- **sleep:** whether participants had slept between learning and the test (no, yes)

- **novel_pic:** whether the picture presented was a previously unseen picture (yes) or the same one used in training (no)
- **item:** the word trained and tested
- **acc:** whether the picture was correctly named (1) or not (0)
- **RT:** the response time for correctly named pictures (ms)

Remember that you can click the grid icon in your enviroment window to see what your new dataset looks like. You should see that it includes one row per trial per participant. You might see that the RT column contains a lot of missing data (*NA*). This is because we were only interested in response time when the answer was correct (acc = 1).

Analysis using means

Let's start by running a traditional analysis on this dataset. We can use a t-test to analyse whether the children were faster to name the pictures after a period of sleep (sleep = yes) or wake (sleep = no).

T-test of participant averages

We can't run a t-test on the dataset as it is because it violates the assumption of independence: we have multiple responses from each participant. Instead, we need to create an average response time for each condition for each participant. You can copy and paste this code to achieve this:

```
ppt_means <- sleep_dat %>%      # take the existing data frame
  group_by(ppt_id, sleep) %>%   # for each value of ppt_id & sleep condition
  summarise(n_trials = sum(!is.na(RT)),  # count valid RT values
            mean_RT = mean(RT, na.rm = TRUE)) %>% # & calculate the mean RT
  pivot_wider(names_from = sleep, names_prefix = "sleep_",
              values_from = c(n_trials, mean_RT)) # reformat into a wide data
```

There are some new data-wrangling functions here that you might find helpful for future reference. You don't need to understand this in great detail for the purpose of this practical, but there's an explainer in the note below if you're interested.

> **i** Data wrangling with the tidyverse package
>
> We have created a new object called `ppt_means` by taking our dataset, *and then* (`%>%`) grouping by participant `ppt_id` and `sleep` condition. *Then* (`%>%`) we feed this to the `summarise()` function to calculate some summary variables for each combination of those groups.
> The first summary variable counts up how many RT values there are to average across (i.e., how many are `!`not missing data (`is.na()`) on this variable. We've called that new variable `n_trials`.
> The second summary variable calculates the mean of the RT values. We've called it `mean_RT`, and specified that it should be calculated using the `mean()` function. We have had to tell it to remove the missing values when doing this (`na.rm = TRUE`).
> Finally, the last line (`pivot_wider()`) rearranges the dataset into wide format, so that we have one participant per row. We've used the values of the sleep variable to name each new column, and specified that those columns should include both the number of trials and mean RT.
> If you're interested in learning how to clean and format your own data, the DataCamp course *Introduction to the Tidyverse*is highly recommended.

Inspect the new object that you have created. You should see that you now have just one row per participant. The first two columns tell us the number of trials used to calculate each participant's average for each sleep condition, and then the last two columns tell us the mean RT for each condition.

What do you make of this? Is it right that we will treat each participant's average score the same in the analysis, even though some are estimated from fewer trials than others?

Now use the `t.test()` function to run a *paired* t-test. Use the internet or the help files to help you. Your output should look as follows:

```
    Paired t-test

data:  ppt_means$mean_RT_sleep_no and ppt_means$mean_RT_sleep_yes
t = 4.5257, df = 30, p-value = 0.00008858
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 303.8399 803.5749
sample estimates:
mean difference
       553.7074
```

> Hint!
>
> You will need to use the `$` to specify the column names, in the format `dataframe$variable`.

```
# Code for paired t-test between two columns
t.test(ppt_means$mean_RT_sleep_no, ppt_means$mean_RT_sleep_yes, paired = TRUE)
```

T-test of word averages

The t-test on participant means tells us that that sleep effect likely generalises to other people in the population. What if we also want to know whether it generalises to other words that we might have chosen to teach?

If you have time, repeat the steps above but for items instead of participants: create a set of *item* means by calculating an average RT for each target and condition. Then run a paired t-test. Are the results consistent with the participant version? **If you've already spent a while doing the above steps, move straight onto the next section.**

```
# Calculate item means
item_means <- sleep_dat %>%     # take the existing data frame
  group_by(item, sleep) %>%     # for each id and condition
  summarise(n_trials = sum(!is.na(RT)),           # count valid trials
            mean_RT = mean(RT, na.rm = TRUE)) %>% # & calculate the mean RT
  pivot_wider(names_from = sleep, names_prefix = "sleep_",
              values_from = c(n_trials, mean_RT)) # reformat into wide format

# Run by-items t-test
t.test(item_means$mean_RT_sleep_no, item_means$mean_RT_sleep_yes, paired = TRUE)
```

Fitting a mixed effects model

Mixed effects models build on the previous approach by allowing us to account for both participant *and* item variation in the same model. We use the original trial-level data so that we are not wasting information by averaging.

Let's start by adding some random intercepts:

```
mod1 <- lmer(RT ~ 1 + sleep + (1|ppt_id) + (1|item), data = sleep_dat)
```

The `lmer()` function fits a linear mixed effects regression model. We write out the formula, which includes:

- The dependent variable we are analysing, `RT`.
- The fixed effects we wanted it to be predicted by (`~`). This includes the intercept (`1`) and the single predictor of `sleep` condition. We could have added others here too—for example, the `novel_pic` variable.
- + the random effects, wrapped in brackets. We specify that we want an intercept (`1`) fitted for each `ppt_id`, and the same again for each `item` tested.

We then specify the dataset that we want this model fitted to. We store the model output as `mod1`. If you have run this model, you can now inspect it by calling the `summary()` function on it.

```
summary(mod1)
```

Take a look through the different parts of the output. It provides information on (1) the model that has been fitted; (2) the residuals; (3) Random effects and number of observations/clusters; and (4) the coefficients and significance tests for the fixed effects.

The fixed effect coefficients tell us about the different conditions in our data. R has automatically added the condition value to our sleep predictor, and so we can see that `sleepyes` is associated with a ~448 *decrease* in reaction time versus the assumed baseline (sleep - no). That is, participants are significantly *faster* at naming pictures if they have slept between learning and test, relative to the baseline condition where they did not sleep.

Because the "sleep - no" condition is being treated as our baseline, the intercept reflects the reaction time in this condition (~2090 ms).

Exploring random effects

Exploring the random coefficients

Our model included random intercepts: that is, it modelled how each participant or item's average response time deviated from the overall mean. We can find out more about this spread by plotting the random effects as follows:

```
dotplot.ranef.mer(ranef(mod1))
```

> Tip!
>
> You might want to increase the size of the plot window (bottom right quadrant), so that you can see the plots more clearly.

You can navigate between the two plots to see which participant IDs and words were fastest (-) and slowest (+) overall, relative to the overall intercept. We're not typically interested in these values but the information comes for free in the modelling processes, and it can be a useful check to see if any of your experimental stimuli are behaving strangely.

If you want a little joy, enter the name of the fastest item into a Google image search to see why children enjoyed learning that word! :)

Adding random slopes

Typically, we also want to consider adding random slopes to model variation in our fixed effects. Do some participants/words show larger sleep effects than others?

Let's start by adding a random slope for the effect of sleep across different items. We do this by adding the predictor alongside the intercept in the random effects part of the formula:

```
mod2 <- lmer(RT ~ sleep + (1|ppt_id) + (1+sleep|item), data = sleep_dat)
```

You should now see that the model lists a variance component for the effect of sleep on the item grouping, and the fixed effect coefficient has changed slightly.

Now create a third model (`mod3`) which *also* includes a random slope for the effect of sleep across different participants (`ppt_id`), and inspect the output. This would reflect the ***maximal model*** in that it includes all possible random slopes.

Model selection

You might have noticed that R printed a warning during model fitting: *boundary (singular) fit: see help('isSingular').* In the summary output, you can see that the intercepts and slopes for ppt_id are perfectly negatively correlated (-1.00). You can see this for yourself if you plot the random effects as we did above—when participants have high intercept values, they have smaller condition values.

We would not expect such perfectly correlated values, and the warning indicates that the model could not be estimated properly. In this situation, we would typically simplify the model (i.e., remove random effects components) until the warning goes away, though there isn't a consensus on the best procedure. This is what we mean when we say the models can be challenging to fit!

Testing model assumptions

In using mixed effects models, we are still fitting a variation of the linear model. Linear models rely on the assumption that the residuals are normally distributed—just as you learned to check for your analyses in Years 1 and 2. We can check this by plotting them:

```
# distribution of residuals
hist(resid(mod2))

# qqplot
qqnorm(resid(mod2))
```

The histogram doesn't look *too* bad, but we can see a slight positive skew: there is a longer tail towards more positive values. We can see this in the QQ plot too: ideally we would like to see a straight line, but it curves off towards higher values, suggesting that the model does not fit this part of the data well.

Sometimes a simple transformation can fix this problem, such as a log-transform. Try fitting another model, but use the log function to transform the dependent variable within the model formula (`log(RT)`). Inspect the residuals as above: what do you think of this new model?

```
log_mod <- lmer(log(RT) ~ sleep + (1|ppt_id) + (1+sleep|item), data = sleep_dat)

hist(resid(log_mod))
qqnorm(resid(log_mod))
```

Summary

Congratulations, you have now fitted a mixed effects model in R! We modelled participant and item effects simultaneously in a psycholinguistic dataset, where this type of analysis has been the dominant approach for the last ~10 years. We interpreted the fixed effects of interest to compare our conclusions to standard t-tests. We also inspected the variability across individual participants and items to understand different components of the models. Finally, we confronted some common issues in model fitting and assumptions.

Challenge yourself

We fitted a linear mixed effects model on response time data, a continuous variable. Mixed effects models are extremely flexible, and we can also use them to model accuracy data (which have binary values of 0 and 1). The underlying maths is different, but the process of fitting them in R is largely the same.

Try to figure out how to analyse the accuracy data available in this dataset. You will need to find the function for running a *binomial* mixed effects model, and specify the family as you run it.

Can you fit a maximal random effects structure for this model?

Further learning

The reading list on the VLE includes the textbook chapter as usual. There is also an excellent tutorial paper by Violet Brown that will help you to consolidate some of the steps we've taken today. If you want to gain more interactive experience in fitting mixed effects models in R, there is a relevant short course on DataCamp.

> ⚠ How's it going?
>
> Let us know how today went via this quick survey - rate the difficulty and length, and leave any comments that would help for us to know (positive comments welcome too!). You will need to be logged in with your university account to respond, but your submission will be anonymous. For more general questions about the practical, please post them on the VLE Discussion Board.